Ranking millas es el único que requiere un heap. Pueden preservar sus otros máximos como atributos y actualizarlos operación a operación. A veces cuando algo es muy restrictivo ese costo fue aprovechado previamente por otra operación que actualizó las estructuras necesarias.

Su ABB de usuarios y su cola de prioridades de millas pueden relacionarse con handles.

Por el orden de complejidad que pide el enunciado para cada método, entendimos que al menos hasta “consultarSaldo/consultarCategoria” la estructura óptima para organizar la información es ABB sobre un diccionario (IDs claves, millas canjeadas, totales y disponibles).
Luego, nos dimos cuenta que sólo con esa estructura no bastaba porque la complejidad de “elMásAcumulador/Canjeador” requerida es O(1) y de “rankingMillas” es O(N Log(U)). Entendemos que para esto la estructura más pertinente son tres heap max con handles que implementan cola por prioridad, lo cual salva el orden de modificación.

Nos cuesta mucho lidiar con la abstracción del problema: sabemos que los IDs se tienen que organizar en un diccionario de ABB (nos da el orden justo para acceder a los usuarios) y que cada vez que se indexe un usuario tenemos que obtener tres handles que apunten a los respectivos heap max de los rankings. No nos queda claro cómo podríamos hacer ese puente, nos perdemos cuando empezamos a formularlo.

En cuanto a las promociones pensamos en construir otro diccionario implementado en trie, cuyas llaves sean los nombres y los valores factor, tope y acumulado. Siendo tope 0 para aquellas promociones sin tope. Es lo más conveniente

AirMiles

Los handles viven dentro de sus estructuras. Tienen que cuidar no romper el encapsulamiento y que no sea posible falsear invariantes.

Atributos privados

- usuarios: dicc(ID: int, Usuario: class). Usuario es una clase privada que contiene la información del usuario: numeroBonos (int), millasTotales, millasCanjeadas y millasDisponibles (ambas double >0).
- Punteros a los rankings: ¿ArrayList(handle)? Suponemos que hay que hacer un handle con tres punteros o tres contenedores distintos.
- ranking_totales_usuarios: cola de prioridad implementada en heap (millas (double)). El orden determinado por millasTotales.

Solo un heap alcanza.

- ranking_disponibles_usuarios: cola de prioridad en heap (millas (double)). El orden determinado por millasDisponibles.
- ranking_canjeadas_usuarios: cola de prioridad en heap (millas (double)). El orden determinado por millasCanjeadas.
- promociones: dicc(NombrePromo: String, Promo: class). Promo es una clase privada que contiene FactorP, acumulado (double >0) y tope (int >0).

Métodos

- void crearSistema(); (O(1))
  → Instancia una promoción “NoP” con factorP = 1.0
  → Instancia los atributos privados vacíos.
  → Sólo modifica promociones.

- void registrarUsuario(int id); (O(Log(U) y O(U) en el peor caso)
  → Crea una entrada nueva (**id**) y vacía en usuarios.
  → Genera el handle y la entrada en los rankings.

- void registrarUsuarioConReferidor(int id, int ref); (O(Log(U) y O(U) en el peor caso)
  → Crea una entrada nueva (**id**) y vacía en el usuarios.
  → Genera el handle y la entrada en los rankings.
  → Le asigna 1 bono al **id** y le suma otro al contador del **ref**

- void acumularMillas(int id, double distancia, String nombrePromo); (O(Log U + |L|) y O(U + |L|) en el peor caso)
  → Decidir si la promo **nombrePromo** aplica o no.
    → Si aplica (ver criterio) la usa y actualiza el tope de dicha promoción. Sino pone el default “NoP”.
  → Calcula la ganancia y le suma un vuelo (**distancia**) al usuario **id**.
  → Modifica las millasTotales del **id**, los handles y los rankings.
  → Modifica promociones acorde si se excede el tope.

- void canjearMillas(int id, double cantidad); (O(Log(U) y O(U) en el peor caso)
  → Resta las millas **cantidad** a las millasDisponibles del **id**.
  → Actualiza los handles y los rankings.

- void transferirMillas(int origen, int destino, double cantidad); (O(Log(U) y O(U) en el peor caso)
  → Resta el monto transferido al **origen**.
  →Cuando tengamos clara la estructura, en algún lado vamos a almacenar transferidores exclusivos.
  → Actualiza handles y rankings.

Bien, es acá. Pueden persistir una lista de pares que se actualiza en este punto, referenciada con handles desde cada usuario para poder acceder en O(1) desde cada usuario y borrar también cuando pierdan exclusividad. Va a ser la mejor implementación. Revisen el taller y la clase de handles.

- double consultarSaldo(int id); (O(Log(U) y O(U) en el peor caso)
  → Accede a las millasDisponibles del **id** (solo lectura).

- String consultarCategoria(int id); (O(Log(U) y O(U) en el peor caso)
  → Accede a las millasDisponibles del **id** (solo lectura) y a partir de ese número con tres comparaciones decide la categoría.

- int[] rankingMillas(int n); (O(N log U))
  → Devuelve una lista de **n** usuarios (o la cantidad de usuarios que haya, en su defecto), ordenada de mayor a menor en función de sus millas disponibles.
  → Para eso puede acceder recursivamente a los máximos del ranking.

- void iniciarPromocion(String nombre, double factor); (O(|L|))
  → Instancia una nueva promoción **nombre** y le asigna un factor de promoción **factor**.

- void iniciarPromocionConTope(String nombre, double factor, double tope); (O(|L|))
  → Instancia una nueva promoción **nombre**, le asigna un factor de promoción **factor** y un **tope** (máximo acumulado que puede usarse).

- void finalizarPromocion(String nombre); (O(|L|))
  → Elimina la promoción **nombre** del diccionario de promociones.

- int elMasAcumulador(); (O(1))

→ Selecciona el usuario que más millas lleva acumuladas al momento que es el máximo del ranking.

Estos dos pueden ser atributos del sistema

- int elMasCanjeador(); (O(1))

→ Selecciona el usuario que más millas lleva acumuladas al momento que es el máximo del ranking.

- int[][] paresTransferidoresExclusivos(); (O(U))

→ Devuelve un arreglo de arreglos (tipo matriz de dimensión n_transferidores exclusivos X 2) en el cual, cada subarreglo es un par de transferidores exclusivos (sin importar el sentido de las transferencias, sólo pueden haber transferido entre sí —con una basta—).

→ Nos queda pendiente ver cómo implementarlo en función a todo lo de arriba.

Les propuse previamente una alternativa superadora.

En general:
Recuerden hacer estructuras de tipos genéricos. El heap es un heap de tipo T que implementa una cola de prioridad. El ABB igual. Internamente podrían tener handles para vincularlos con otras estructuras que lo requieran, pero siempre pensando en genérico, cuidando modularidad e invariante sin exponer atributos ni métodos privados.
Separen en clases. Usuario, promoción, las de las estructuras básicas y todas las que consideren.

Su usuario podría tener más atributos si los necesitaran más adelante.